

many cases. We can also check multiple expressions by separating them using commas:

```
number := 10
switch number
{
    case 5:
        fmt.Println("The number is 5")
    case 10,20,30:
        fmt.Println("The number is a multiple of 10")
    case 7,13:
        fmt.Println("The number is prime!")
}
```

Loops

The *for* loop is:

```
func main()
{
    for i := 1; i <= 5; i++
    {
        fmt.Println("Value:", i)
    }
}
```

Multiple initialisation and increment in a *for* loop is as follows:

```
package main
import "fmt"
func main()
{
    for no, i := 10, 1; i <= 10 && no <= 19; i, no = i+1, no+1
    { //multiple initialisation and increment
        fmt.Printf("%d * %d = %d\n", no, i, no*i)
    }
}
```

The syntax for creating an infinite loop is shown below:

```
for
{
}
```

Functions in Go

The syntax of function definition in the Go programming language is as follows:

```
func function_name( [parameter list] ) [return_types]
{
    body of the function
}
```

The function declaration starts with the *func* keyword followed by the function's name. The parameters are specified between ('and') followed by the *returntype* of the function. The syntax for specifying a parameter is the parameter's name followed by the type. Any number of parameters can be specified like *parameter1 type, parameter2 type*. Then the body of the function block, or collection of statements that define what the function does, is enclosed in { }.

```
func sumTotal(price, no int) int //if the consecutive
parameters are of the same
```

```
{
    //type, we can skip writing the type
each time
    var total = price * no
    return total
}
```

```
package main
import "fmt"
func calculateBill(price, num int) int
{
    var totalPrice = price * num
    return totalPrice
}
func main()
{
    price, num := 90, 6
    totalPrice := calculateBill(price, num)
    fmt.Println("Total price is", totalPrice)
}
```

Function to swap 2 numbers:

```
func swap(x, y int) (int, int) {
    return y, x
}
```

```
func main() {
    a, b := swap(10, 20)
    fmt.Println(a, b)
}
```

Arrays, slices and maps in Go

Instead of declaring individual variables, such as *number0*, *number1...* and *number99*, you declare one array variable such as 'numbers' and use *numbers[0]*, *numbers[1]*, and *numbers[99]* to represent individual variables. In Go, an array is a numbered sequence of elements of a specific length.

```
var variable_name [SIZE] variable_type
Eg: var x [10] int //This is the example of array
//declaration in go

func main() {
    var a [2]string
```